Cloud-containerization-and-DevOps

Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

Part 1: Docker Volumes - Persistent Data Storage

Lab 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

```
# Create a container that writes data
docker run -it --name test-container ubuntu /bin/bash

# Inside container:
echo "Hello World" > /data/message.txt
cat /data/message.txt  # Shows "Hello World"
exit

# Restart container
docker start test-container
docker exec test-container cat /data/message.txt
# ERROR: File doesn't exist! Data was lost.
```

> Solution: Docker Volumes

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -it --name test-container
ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
817807f3c64e: Pull complete
e4a1e8de092c: Download complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072fee
f0c
Status: Downloaded newer image for ubuntu:latest
root@15da38de8f32:/#
```

```
root@15da38de8f32:/# mkdir /data
root@15da38de8f32:/# echo "Hello World" > /data/message.txt
root@15da38de8f32:/# cat /data/message.txt
Hello World
root@15da38de8f32:/# exit
exit
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker start test-container
test-container
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker exec test-container cat /data/
message.txt
Hello World
```

Lab 2: Volume Types

1. Anonymous Volumes

```
# Create anonymous volume (auto-generated name)
docker run -d -v /app/data --name web1 nginx

# Check volume
docker volume ls
# Shows: anonymous volume with random hash

# Inspect container to see volume mount
docker inspect web1 | grep -A 5 Mounts
```

2. Named Volumes

```
# Create named volume
docker volume create mydata

# Use named volume
docker run -d -v mydata:/app/data --name web2 nginx

# List volumes
docker volume ls
# Shows: mydata

# Inspect volume
docker volume inspect mydata
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker volume create mydata
mydata
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
--name web2 \
-v mydata:/app/data \
nginx
32b1e567b876c41ebda73f837bad5f427250cf6516033bcd9c8a1ac193898ce3
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker volume ls
DRIVER    VOLUME NAME
local     00cbe078c61bece7062e787243a4f79685e2d22b86462fd9a0d76d8630c777f8
local     6fc2dbc37ecbadeab75df0d357f7c75376d351ac8427a8dfc4767871f080be9d
local     38d08c83a4e91a9437770b43beae89a8c339464e98cd4ed4f0d064fcf860473b
local     332eec9471b31c0c62578961487f38ae027142e49c286c9bbd02228ba085e0b5
local     assignment-1_pgdata
local     dbdata
local     k3d-mycluster-images
local     mydata
local     myvolume
local     test
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker volume inspect mydata
[
    {
        "CreatedAt": "2026-03-22T15:29:37Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
        "Name": "mydata",
        "Options": null,
        "Scope": "local"
    }
]
```

3. Bind Mounts (Host Directory)

```
# Create directory on host
mkdir ~/myapp-data

# Mount host directory to container
docker run -d -v ~/myapp-data:/app/data --name web3 nginx

# Add file on host
echo "From Host" > ~/myapp-data/host-file.txt

# Check in container
docker exec web3 cat /app/data/host-file.txt
# Shows: From Host
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ mkdir ~/myapp-data
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
--name web3 \
-v ~/myapp-data:/app/data \
nginx
e997985e0420e327087669998fbf9e8918e698dfeaf9ebe25e032d1fb24ee648
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ echo "From Host" > ~/myapp-data/test.
txt
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker exec web3 cat /app/data/test.t
xt
From Host
```

—

Lab 3: Practical Volume Examples

Example 1: Database with Persistent Storage

```
# MySQL with named volume
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0

# Check data persists
docker stop mysql-db
docker rm mysql-db

# New container with same volume
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
# Data is preserved!
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
d4c7048d1cf1: Pull complete
771773a1fe7a: Pull complete
6375ec02106f: Pull complete
5915b0e5580c: Pull complete
60c3210ab1f6: Pull complete
c3b8cfb10b41: Pull complete
440208b3f29f: Pull complete
1378718ee973: Pull complete
10ee8ef3b9fc: Pull complete
cd8097e7c26b: Pull complete
0940ecf5daec: Pull complete
de0490608ca2: Download complete
2f1dcce71f1f: Download complete
Digest: sha256:64756cc92f707eb504496d774353990bcb0f6999ddf598b6ad188f2da66bd
000
Status: Downloaded newer image for mysql:8.0
d3becb7d728089f8193afa53f60499054c946e19ee17f0c28f5ee6b7c1a24685
```


```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
d4c7048d1cf1: Pull complete
771773a1fe7a: Pull complete
6375ec02106f: Pull complete
5915b0e5580c: Pull complete
60c3210ab1f6: Pull complete
c3b8cfb10b41: Pull complete
440208b3f29f: Pull complete
1378718ee973: Pull complete
10ee8ef3b9fc: Pull complete
cd8097e7c26b: Pull complete
0940ecf5daec: Pull complete
de0490608ca2: Download complete
2f1dcce71f1f: Download complete
Digest: sha256:64756cc92f707eb504496d774353990bcb0f6999ddf598b6ad188f2da66bd
000
Status: Downloaded newer image for mysql:8.0
d3becb7d728089f8193afa53f60499054c946e19ee17f0c28f5ee6b7c1a24685
```


Example 2: Web App with Configuration Files

```
# Create config directory
mkdir ~/nginx-config

# Create nginx config file
echo 'server {
    listen 80;
    server_name localhost;
    location / {
```

```
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf

# Run nginx with config bind mount
docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx

# Test
curl http://localhost:8080
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker stop mysql-db
docker rm mysql-db
mysql-db
mysql-db
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
8c5cf44a9167df443a4b0c6939e577b1417f3e93ba2cfe68c1926a96201848f5
```

—

Lab 4: Volume Management Commands

```
# List all volumes
docker volume ls

# Create a volume
docker volume create app-volume

# Inspect volume details
docker volume inspect app-volume

# Remove unused volumes
docker volume prune

# Remove specific volume
docker volume rm volume-name

# Copy files to/from volume
docker cp local-file.txt container-name:/path/in/volume
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker volume ls
DRIVER    VOLUME NAME
local     00cbe078c61bece7062e787243a4f79685e2d22b86462fd9a0d76d8630c777f8
local     6fc2dbc37ecbadeab75df0d357f7c75376d351ac8427a8dfc4767871f080be9d
local     38d08c83a4e91a9437770b43beae89a8c339464e98cd4ed4f0d064fcf860473b
local     332eec9471b31c0c62578961487f38ae027142e49c286c9bbd02228ba085e0b5
local     app-volume
local     assignment-1_pgdata
local     dbdata
local     k3d-mycluster-images
local     mydata
local     mysql-data
local     myvolume
local     test
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker volume create app-volume
app-volume
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker volume inspect app-volume
[
    {
        "CreatedAt": "2026-03-27T06:07:18Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
        "Name": "app-volume",
        "Options": null,
        "Scope": "local"
    }
]
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one c
ontainer.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker volume rm volume-name
Error response from daemon: volume volume-name not found
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker volume rm app-volume
app-volume
```

Part 2: Environment Variables

Lab 1: Setting Environment Variables

Method 1: Using -e flag

```
# Single variable
docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  -p 3000:3000 \
  my-node-app

# Multiple variables
```

```
docker run -d \
  -e VAR1=value1 \
  -e VAR2=value2 \
  -e VAR3=value3 \
  my-app
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  -p 3000:3000 \
  my-node-app
8d280d6382738203483169 5d80f7e1ce56873e6c3f97c45daa7ff0207ad48aca
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
  -e VAR1=value1 \
  -e VAR2=value2 \
  -e VAR3=value3 \
  my-node-app
59c09dd25b9943f21dd4a516b9052c6888472f77bb1e343c67bcbefd394bb73a
```

Lab 2: Environment Variables in Applications

Python Flask Example

```python
# app.py
import os
from flask import Flask

app = Flask(__name__)

# Read environment variables
db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/flask-env-app$ cat app.py
import os
from flask import Flask

app = Flask(__name__)

# Read environment variables
db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

Dockerfile with Environment Variables

```
FROM python:3.9-slim

# Set environment variables at build time
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

# Default runtime environment variables
ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000

CMD ["python", "app.py"]
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/flask-env-app$ cat Dockerfile
FROM python:3.9-slim

ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000

CMD ["python", "app.py"]
```

Lab 3: Test Environment Variables

```
# Run with custom env vars
docker run -d \
  --name flask-app \
  -p 5000:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e PORT="8080" \
  flask-app

# Check environment in running container
docker exec flask-app env
docker exec flask-app printenv DATABASE_HOST

# Test the endpoint
curl http://localhost:5000/config
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
  --name flask-app \
  -p 5000:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e PORT="5000" \
  flask-app
8fa6b865af4ca513e56656431242975381e7f0e38f887826590353febc1bb0b6
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/flask-docker$ docker exec flask-app en
v
PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:
/bin
HOSTNAME=149d273a6168
```

—

Part 3: Docker Monitoring

Lab 1: Basic Monitoring Commands

`docker stats` - Real-time Container Metrics

```
# Live stats for all containers
docker stats


# Live stats for specific containers
docker stats container1 container2

# Specific format output
docker stats --format "table \t\t\t"

# No-stream (single snapshot)
docker stats --no-stream

# All containers (including stopped)
docker stats --all
```

Useful Format Options:

```
# Custom format
docker stats --format "Container:  | CPU:  | Memory: "

# JSON output
docker stats --format json --no-stream

# Wide output
docker stats --no-stream --no-trunc
```

Lab 2: `docker top` - Process Monitoring

```
# View processes in container
docker top container-name

# View with full command line
docker top container-name -ef

# Compare with host processes
ps aux | grep docker
```

Lab 3: `docker logs` - Application Logs

```
# View logs
docker logs container-name

# Follow logs (like tail -f)
docker logs -f container-name

# Last N lines
docker logs --tail 100 container-name

# Logs with timestamps
docker logs -t container-name

# Logs since specific time
docker logs --since 2024-01-15 container-name

# Combine options
docker logs -f --tail 50 -t container-name
```

Lab 4: Container Inspection

```
# Detailed container info
docker inspect container-name

# Specific information
docker inspect --format='' container-name
docker inspect --format='' container-name
docker inspect --format='' container-name

# Resource limits
docker inspect --format='' container-name
docker inspect --format='' container-name
```

Lab 5: Events Monitoring

```
# Monitor Docker events in real-time
docker events

# Filter events
docker events --filter 'type=container'
docker events --filter 'event=start'
docker events --filter 'event=die'

# Since specific time
docker events --since '2024-01-15'

# Format output
docker events --format '  '
```

Lab 6: Practical Monitoring Script

```
#!/bin/bash
# monitor.sh - Simple Docker monitoring

echo "=== Docker Monitoring Dashboard ==="
echo "Time: $(date)"
echo

echo "1. Running Containers:"
docker ps --format "table \t\t"
echo

echo "2. Resource Usage:"
```

```
docker stats --no-stream --format "table \t\t\t\t"
echo

echo "3. Recent Events:"
docker events --since '5m' --until '0s' --format '  ' | tail -5
echo

echo "4. System Info:"
docker system df
```

Part 4: Docker Networks

Lab 1: Understanding Docker Network Types

List Networks

```
# Default networks
docker network ls

# Output:
# NETWORK ID     NAME      DRIVER    SCOPE
# abc123         bridge    bridge    local
# def456         host      host      local
# ghi789         none      null      local
```

Lab 2: Network Types Explained

1. Bridge Network (Default)

```
# Containers on bridge network can communicate
# Each container gets own IP, isolated from host

# Create custom bridge network
docker network create my-network

# Inspect network
docker network inspect my-network

# Run containers on custom network
docker run -d --name web1 --network my-network nginx
```

```
docker run -d --name web2 --network my-network nginx

# Containers can communicate using container names
docker exec web1 curl http://web2
```

2. Host Network

```
# Container uses host's network directly
# No network isolation, shares host's IP

docker run -d --name host-app --network host nginx

# Access directly on host port 80
curl http://localhost
```

3. None Network

```
# No network access
docker run -d --name isolated-app --network none alpine sleep 3600

# Test - no network interfaces
docker exec isolated-app ifconfig
# Only loopback interface
```

4. Overlay Network (Swarm)

```
# For Docker Swarm - multi-host networking
docker network create --driver overlay my-overlay
```

—

Lab 3: Network Management Commands

```
# Create network
docker network create app-network
docker network create --driver bridge --subnet 172.20.0.0/16 --gateway 172.20.0.1 my-subne

# Connect container to network
docker network connect app-network existing-container

# Disconnect container from network
docker network disconnect app-network container-name
```

```
# Remove network
docker network rm network-name

# Prune unused networks
docker network prune
```

—

Lab 4: Multi-Container Application Example

Web App + Database Communication

```
# Create network
docker network create app-network

# Start database
docker run -d \
  --name postgres-db \
  --network app-network \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15

# Start web application
docker run -d \
  --name web-app \
  --network app-network \
  -p 8080:3000 \
  -e DATABASE_URL="postgres://postgres:secret@postgres-db:5432/mydb" \
  -e DATABASE_HOST="postgres-db" \
  node-app

# Web app can connect to database using "postgres-db" hostname
```

Lab 5: Network Inspection & Debugging

```
# Inspect network
docker network inspect bridge

# Check container IP
docker inspect --format='' container-name

# DNS resolution test
docker exec container-name nslookup another-container
```

```
# Network connectivity test
docker exec container-name ping -c 4 google.com
docker exec container-name curl -I http://another-container

# View network ports
docker port container-name
```

Lab 6: Port Publishing vs Exposing

```
# PORT PUBLISHING (host:container)
docker run -d -p 80:8080 --name app1 nginx
# Host port 80 → Container port 8080

# Dynamic port publishing
docker run -d -p 8080 --name app2 nginx
# Docker assigns random host port

# Multiple ports
docker run -d -p 80:80 -p 443:443 --name app3 nginx

# Specific host IP
docker run -d -p 127.0.0.1:8080:80 --name app4 nginx

# EXPOSE in Dockerfile (metadata only)
# Dockerfile: EXPOSE 80
# Still need -p to publish
```

Part 5: Complete Real-World Example

Application Architecture:

- Flask Web App (port 5000)
- PostgreSQL Database (port 5432)
- Redis Cache (port 6379)
- All connected via custom network

Implementation:

```
# 1. Create network
docker network create myapp-network

# 2. Start database with volume
docker run -d \
  --name postgres \
  --network myapp-network \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e POSTGRES_DB=mydatabase \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:15

# 3. Start Redis
docker run -d \
  --name redis \
  --network myapp-network \
  -v redis-data:/data \
  redis:7-alpine

# 4. Start Flask app with all configurations
docker run -d \
  --name flask-app \
  --network myapp-network \
  -p 5000:5000 \
  -v $(pwd)/app:/app \
  -v app-logs:/var/log/app \
  -e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
  -e REDIS_URL="redis://redis:6379" \
  -e DEBUG="false" \
  -e LOG_LEVEL="INFO" \
  --env-file .env.production \
  flask-app:latest
```

Monitoring Commands:

```
# Check all components
docker ps

# Monitor resources
docker stats postgres redis flask-app

# Check logs
docker logs -f flask-app

# Network connectivity test
docker exec flask-app ping -c 2 postgres
docker exec flask-app ping -c 2 redis
```

```
# View network details
docker network inspect myapp-network
```

—

conclusion

1. **Volumes** persist data beyond container lifecycle
2. **Environment variables** configure containers dynamically
3. **Monitoring commands** help debug and optimize containers
4. **Networks** enable secure container communication
5. **Always use named volumes** for production data
6. **Custom networks** provide better isolation and DNS
7. **Monitor resource usage** to prevent issues
8. **Use .env files** for sensitive configuration